

46-926, Fall 2020: Homework #3

Due 3:20pm Eastern, November 18, 2020.

Homework should be submitted electronically on canvas before the deadline. Please submit your homework as a single file (pdf preferred, canvas struggles with html). We recommend using jupyter notebooks to prepare your homework, calling R code through rpy2.

Do all computational problems in python unless otherwise noted. Please post any questions on the Piazza discussion board.

1. *The orthogonal case.* We're going to consider a special case where we can actually write out the solution to both ridge regression and the lasso in simple, closed form.

Consider the usual regression setup with n observations on p variables, with X, Y, β being the usual matrix and vectors. However, assume the very special setup where X is an orthogonal matrix (orthogonal, length 1 columns, such that $X^T X = I$). Then it turns out that solving ridge regression and the lasso are equivalent to solving the following two problems

$$\begin{aligned}\hat{\beta}_{ridge} &= \underset{\beta}{\operatorname{argmin}} \|X^T Y - \beta\|_2^2 + \lambda \|\beta\|_2^2 \\ \hat{\beta}_{lasso} &= \underset{\beta}{\operatorname{argmin}} \|X^T Y - \beta\|_2^2 + \lambda \|\beta\|_1\end{aligned}$$

I encourage you to think about why this is true; it is closely tied to the picture on Lecture 1, slide 19 (and related to the surrounding discussion). However, you can just assume that it is true for this problem.

Note: This problem is new, so there is always a chance that I might have a mistake or should have given a better hint somewhere. If something feels wrong, drop me a note in Piazza before you lose too much time.

- (a) Show that you can rewrite each optimization problem in the form

$$\underset{\beta}{\operatorname{argmin}} \sum_{j=1}^p \text{something}_j,$$

such that it is sufficient to minimize each term separately over β_j .

- (b) Compute the minimizer $\hat{\beta}_j$ for both cases. This gives you the closed-form solution to ridge regression *and* the lasso!

Hints:

In the end, you want to find $\hat{\beta}_j$ as a function of $(X^T Y)_j$.

For the ridge regression case, one approach is simple differentiation.

For the lasso, you have the issue of non-differentiability at 0 for the absolute value. As a result, your answer will probably be a piecewise function, broken into cases with different behavior depending on z_i . You might find it useful to begin by considering the behavior of the objective in the cases where $\beta_j > 0$ and $\beta_j < 0$, and the implications for the minimizer.

- (c) Plot the solution $\hat{\beta}_j$ as a function of $(X^T Y)_j$ for both ridge regression and the lasso.
- (d) At least for this special, orthogonal case, can you give a simple explanation for why ridge regression effectively has a much bigger impact on large coefficients, and lasso effectively has a much bigger impact on smaller coefficients?
2. *Prediction error and dimension.* Two homeworks ago, you explored the role of dimension in the prediction error of linear regression. In this problem, you will extend that simulation to include the lasso.

Note: Do this problem in python.

Carry out essentially the same regression simulation as last homework (feel free to modify the same code, or the code from the solutions), but this time you will consider two different predictors:

- (a) Linear regression, as you did last time
- (b) Lasso regression, with regularization parameter $\sqrt{\frac{\log p}{50n}}$. (This way you don't have to cross-validate, just use this regularization parameter.)

Each time through the inner loop (or equivalent) of your simulation, you will fit both models, and compute the average squared prediction error for both estimates.

Just like last time, use $n = 100$ and average 100 samples for each size. Just do $p = 2, \dots, 80$, $p = 1$ will lead to a bit of odd behavior.

Plot both of the average squared prediction errors against p (on the same plot). Make sure to use colors or symbols to distinguish between the models, and add a legend.

What is the conclusion of your experiment – how do the methods compare?

Hints:

In python, use the Lasso function from `sklearn.linear_model`. This is called like

```
model = Lasso(alpha=[What you want], normalize=True,
               fit_intercept=True)
model.fit(X_train, y_train)
#Then call model.predict, etc.
```

where `alpha` is our λ .

3. In this problem, we will explore ideas related to a recent paper, Preis et al. (2013), which attempts to model DJIA returns based on Google Trends data.

In `trends_train.csv` and `trends_test.csv`, you will find training and test data sets of matching column format. For now we'll work with the training data set.

The data sets consist of the following columns:

- The date of the Sunday that *ends* the week of google trends data, which is aggregated Monday - Sunday. The volume differences in the other columns start one week before this date and end on this date.
- Google trends data for 96 words, with words selected by the methodology in the paper. The time series for each word reflects *change in* the volume of search queries that week, compared to the previous week. The original time series (before taking differences) were normalized to have a maximum of 100 over time; you can find these in `trendtimeseries.csv` just for fun.
- The weekly DJIA log returns for the week *after* the google trends data (Monday - Monday adjusted closing price).

You should do this problem in R with `glmnet`, you will find it significantly easier.

See the demo code from last class for `glmnet` syntax. I suggest starting this problem early in case you have software issue with `rpy2`.

Preis, T., Moat, H. S., & Stanley, H. E. (2013). Quantifying trading behavior in financial markets using Google Trends. Scientific reports, 3, srep01684.

- (a) Load the training data and see what you have. Fit the lasso to predict the weekly log return from the weekly changes in search volume. Use the default range of penalization parameters. Plot the coefficients as a function of $\log(\lambda)$.

- (b) Carry out cross-validation for the lasso (using the built in procedure). Plot the cross-validation curve with its standard errors.

Note: For the purpose of this problem, we're going to carry out cross-validation as if the samples were i.i.d., neglecting the temporal structure. In reality, this would be something to worry about.

- (c) Look at the models that you obtain through each selection rule (minimization vs. 1 s.e.). How many nonzero coefficients does each model have (other than the intercept)? What does this suggest to you about the amount of signal in the problem?

Note: The output of cross-validation depends on the random splits that you select. In this problem, you should get different values for $\lambda_{\min}$ and $\lambda_{1s.e.}$ to make it interesting. If by some chance you select the same model both ways, just rerun cross-validation again or change your seed. (NOTE: This is just to make a nice homework problem, it's not a good practice in general. We will discuss further in class.)

- (d) Let's try the $\lambda_{\min}$ model out. Make predictions on your training data by using the predict function and passing your training X in as the **newx** parameter. The **s** parameter is the λ value you want to fit at. Make a scatterplot of your fitted values against the true values; is there any pattern?
- (e) We could carry out a simple (and overly simplified) trading strategy based on your model, as outlined in the paper. Each Sunday, we look at the prediction from your model. If the log returns for the next week are predicted to be positive, we buy on the Monday and sell on the following Monday. If the log returns are predicted to be negative, we do the opposite. This lets us realize log returns for each week of

$$(\text{sign of our prediction}) \cdot (\text{DJIA log return})$$

which is easy to work with. Compute your return for this strategy on the training data, using your $\lambda_{\min}$ model. Compare it to the return you would have realized if you bought on the first day of the data set and sold on the last day.

- (f) Now apply your $\lambda_{\min}$ model, trained on the training set, to your test data. Again, you just use the predict function, changing the **newx** parameter. Plot your fitted values as before.

Carry out the same simple strategy on the test data. How well do you do? Again, compare to the overall return for the DJIA in this period. In the end, would you have been better off with the $\lambda_{1s.e.}$ model?